

DAM

PROMETEO  
by thePower

# Tema 0: Kotlin

Programación de Servicios y Procesos · 2º DAM

---

Olga M. Moreno Martín

10 / 09 / 2026

SECCIÓN 1

Kotlin

01

# El lenguaje Kotlin

Kotlin es un lenguaje de programación moderno, conciso y seguro que se ejecuta en la máquina virtual de Java (JVM) y también se puede compilar a JavaScript o nativo. Fue desarrollado por JetBrains y Google en 2011 y se ha convertido en el lenguaje de programación oficial para el desarrollo de aplicaciones Android.



# Características de Kotlin

- **Interoperabilidad con Java:** Kotlin es 100% interoperable con Java, lo que significa que puedes usar todas las bibliotecas de Java en tus proyectos de Kotlin y viceversa.
- **Seguridad nula:** Kotlin tiene un sistema de tipos que elimina la posibilidad de errores de puntero nulo en tiempo de ejecución.
- **Concisión y legibilidad:** Kotlin es un lenguaje conciso y fácil de leer. Puedes escribir menos código y hacer más cosas.
- **Programación funcional:** Kotlin admite programación funcional y orientada a objetos. Puedes escribir funciones de orden superior, funciones lambda y mucho más.
- **Extensiones de funciones:** Kotlin te permite agregar nuevas funciones a las clases existentes sin heredar de ellas.
- **Clases de datos:** Kotlin tiene una sintaxis especial para crear clases de datos que contienen solo datos y no tienen comportamiento.
- **Corrutinas:** Kotlin tiene soporte para corrutinas, que te permiten escribir código asíncrono de manera secuencial.
- **Jetpack Compose:** Kotlin es el lenguaje oficial para el desarrollo de aplicaciones Android con Jetpack Compose, un marco de trabajo moderno para la creación de interfaces de usuario.

SECCIÓN 2

# Tipos de variables y datos

En Kotlin, las variables pueden ser explícitas e implícitas, pero cuidado con la inferencia de tipos...

# Tipos de variables y datos

En Kotlin las variables pueden declararse de dos formas, de forma explícita o de forma implícita. En el caso de las variables explícitas, se debe indicar el tipo de dato que almacenará la variable, mientras que en las variables implícitas, el tipo de dato se infiere automáticamente por el compilador.

## ⚠ Sobre la inferencia de tipos

*Que el tipo de dato se infiera automáticamente no quiere decir que Kotlin sea un lenguaje de tipado dinámico, ya que una vez que se asigna un tipo de dato a una variable, no se puede cambiar.*

# Variables explícitas

Para declarar una variable de forma explícita en Kotlin, se debe indicar el tipo de dato que almacenará la variable seguido del nombre de la variable y opcionalmente de su valor inicial.

```
val nombre: String = "Ejemplo"
```

```
val edad: Int = 25
```

En el ejemplo anterior, se declaran dos variables de forma explícita, una de tipo String llamada nombre y otra de tipo Int llamada edad.

# Variables implícitas

Para declarar una variable de forma implícita en Kotlin, se debe utilizar la palabra clave `val` o `var` seguida del nombre de la variable y opcionalmente de su valor inicial. En este caso, el tipo de dato se infiere automáticamente por el compilador.

```
val nombre = "Ejemplo"
```

```
val edad = 25
```

“Si algo es blanco sigue siendo blanco”

# Tipos de datos primitivos

En Kotlin, los tipos de datos primitivos son los mismos que en Java, pero con algunas diferencias en la forma en que se declaran.

- **Byte:** Almacena números enteros de 8 bits.
- **Short:** Almacena números enteros de 16 bits.
- **Int:** Almacena números enteros de 32 bits.
- **Long:** Almacena números enteros de 64 bits.
- **Float:** Almacena números de punto flotante de 32 bits.
- **Double:** Almacena números de punto flotante de 64 bits.
- **Char:** Almacena caracteres Unicode de 16 bits.
- **Boolean:** Almacena valores booleanos (true o false)

`val` entero: Int = 10

`val` flotante: Float = 10.5f

`val` caracter: Char = 'A'

`val` booleano: Boolean = true

# Tipos de datos compuestos

Además de los tipos de datos primitivos, Kotlin también tiene tipos de datos compuestos que permiten almacenar colecciones de datos.

- **Array:** Almacena una colección de elementos del mismo tipo.
- **List:** Almacena una colección de elementos ordenados.
- **Set:** Almacena una colección de elementos únicos.
- **Map:** Almacena una colección de pares clave-valor.

```
val numeros = arrayOf(1, 2, 3, 4, 5)
val nombres = listOf("Juan", "María", "Pedro")
val colores = setOf("Rojo", "Verde", "Azul")
val edades = mapOf("Juan" to 25, "María" to 30, "Pedro" to 35)
```

# Conversión de tipos

En Kotlin, la conversión de tipos se realiza de forma segura y explícita utilizando funciones específicas para cada tipo de dato.

```
val numero: Int = 10
```

```
val texto: String = numero.toString()
```

```
val texto: String = "10"
```

```
val numero: Int = texto.toInt()
```

*En el primer ejemplo, se convierte un número entero a una cadena de texto utilizando la función `toString()`. En el segundo ejemplo, se convierte una cadena de texto a un número entero utilizando la función `toInt()`.*

# Sobre las variables mutables e inmutables

En Kotlin, las variables se pueden declarar como `val` (inmutables) o `var` (mutables). Las variables inmutables no pueden cambiar su valor una vez asignado, mientras que las variables mutables pueden cambiar su valor en cualquier momento.

```
val nombre: String = "Ejemplo" // Variable inmutable  
var edad: Int = 25 // Variable mutable
```

En el ejemplo anterior, la variable `nombre` es inmutable, por lo que su valor no puede cambiar una vez asignado. La variable `edad`, en cambio, es mutable, por lo que su valor puede cambiar en cualquier momento.

# Sobre la mutabilidad y la inmutabilidad

La inmutabilidad es una característica importante en Kotlin, ya que ayuda a prevenir los errores de programación al evitar que los valores de las variables cambien de forma inesperada. Al utilizar variables inmutables, se puede escribir código más seguro y predecible, lo que facilita la depuración y el mantenimiento del código.

Cuándo trabajamos con listas o arrays, podemos modificar los elementos de la lista, pero no podemos cambiar la referencia de la lista.

```
val lista = mutableListOf(1, 2, 3, 4, 5)
lista[0] = 10 // Modifica el elemento en la posición 0
lista = mutableListOf(6, 7, 8, 9, 10) // Error de compilación
```

# Sobre la mutabilidad y la inmutabilidad

Podemos ver también un ejemplo de lista mutable pero cuyo contenido no puede ser modificado.

```
val lista = listOf(1, 2, 3, 4, 5)  
lista[0] = 10 // Error de compilación
```

Esto nos deja claro que la inmutabilidad no solo se refiere a la variable en sí, sino también a los elementos que contiene la variable.

Podemos entonces tener cuatro situaciones posibles:

- **Variable inmutable y elementos inmutables:** No se puede cambiar ni la variable ni los elementos.
- **Variable inmutable y elementos mutables:** No se puede cambiar la variable, pero sí los elementos.
- **Variable mutable y elementos inmutables:** Se puede cambiar la variable, pero no los elementos.
- **Variable mutable y elementos mutables:** Se puede cambiar tanto la variable como los elementos.

# La ventaja de las expresiones en la programación funcional

En la programación funcional, se fomenta el uso de expresiones en lugar de sentencias, ya que las expresiones son más concisas, legibles y fáciles de entender.

Al utilizar expresiones en lugar de sentencias, se puede escribir menos código y hacer más cosas.

Esto es especialmente útil en Kotlin, que es un lenguaje de programación funcional que fomenta el uso de expresiones para realizar operaciones y producir resultados de forma concisa y eficiente..



# Las variables inmutables y la programación funcional

Las variables inmutables son una característica fundamental de la programación funcional, ya que permiten escribir código más seguro y predecible al evitar los efectos secundarios y las mutaciones de estado.

Al utilizar variables inmutables, se puede escribir código más conciso, legible y mantenible, lo que facilita la depuración y el mantenimiento del código.



## SECCIÓN 2

# Expresiones vs Sentencias

En Kotlin, las expresiones y las sentencias son dos conceptos fundamentales que se utilizan para definir el flujo de control y la lógica de un programa. Aunque ambos se utilizan para realizar operaciones y tomar decisiones en un programa, existen diferencias importantes entre ellos que es importante comprender.

# Expresiones

En Kotlin, una expresión es una combinación de valores, variables, operadores y funciones que se evalúa para producir un resultado. Las expresiones pueden ser tan simples como una constante o tan complejas como una llamada a una función que devuelve un valor.

```
val numero = 10
```

```
val resultado = numero * 2
```

Las expresiones en Kotlin pueden tener un valor de retorno, lo que significa que pueden ser utilizadas en cualquier lugar donde se espere un valor, como en la inicialización de una variable, en una sentencia de control de flujo o en una llamada a una función.

# Sentencias

En Kotlin, una sentencia es una instrucción que realiza una acción en un programa. Las sentencias pueden ser tan simples como una asignación de variable o tan complejas como una estructura de control de flujo que toma decisiones basadas en condiciones.

Las sentencias en Kotlin no tienen un valor de retorno, lo que significa que no pueden ser utilizadas en lugares donde se espere un valor, como en la inicialización de una variable o en una llamada a una función.

```
val numero = 10
if (numero > 0) {
    println("El número es positivo")
} else {
    println("El número es negativo")
}
```

# Diferencias entre expresiones y sentencias

Las principales diferencias entre expresiones y sentencias en Kotlin son las siguientes:

- **Valor de retorno:** Las expresiones tienen un valor de retorno y pueden ser utilizadas en lugares donde se espere un valor, mientras que las sentencias no tienen un valor de retorno y no pueden ser utilizadas en lugares donde se espere un valor.
- **Complejidad:** Las expresiones pueden ser tan simples como una constante o tan complejas como una llamada a una función que devuelve un valor, mientras que las sentencias realizan acciones en un programa y pueden ser tan simples como una asignación de variable o tan complejas como una estructura de control de flujo.
- **Uso:** Las expresiones se utilizan para realizar operaciones y producir resultados, mientras que las sentencias se utilizan para realizar acciones en un programa, como asignar valores a variables, tomar decisiones basadas en condiciones o repetir bloques de código.

## Ejemplo de una función que usa sentencias (statement) y expresiones (expression)



Pongamos el ejemplo de una función que dictamina si una persona es mayor de edad o no.

```
fun esMayorDeEdad(edad: Int): Boolean {  
    if (edad >= 18) {  
        return true  
    } else {  
        return false  
    }  
}  
//Statement
```

```
fun esMayorDeEdad(edad: Int): Boolean = edad >= 18  
//Expression
```

SECCIÓN 4

# Funciones y Lambdas

04

En Kotlin, las funciones son ciudadanos de primera clase, lo que significa que puedes tratarlas como cualquier otro tipo de dato, como un Int o un String. Esto te permite pasar funciones como argumentos a otras funciones, devolver funciones de otras funciones y almacenar funciones en variables.

# Funciones

Las funciones en Kotlin se definen utilizando la palabra clave **fun**, seguida del nombre de la función, los parámetros de entrada y el tipo de retorno.

```
fun suma(a: Int, b: Int): Int {  
    return a + b  
}
```

# Funciones de orden superior

En Kotlin, puedes pasar funciones como argumentos a otras funciones. Estas funciones se conocen como funciones de orden superior y te permiten escribir código más conciso y reutilizable.

```
fun operacion(a: Int, b: Int, funcion: (Int, Int) -> Int): Int {  
    return funcion(a, b)  
}  
fun suma(a: Int, b: Int): Int {  
    return a + b  
}  
fun resta(a: Int, b: Int): Int {  
    return a - b  
}  
val resultadoSuma = operacion(10, 5, ::suma)  
val resultadoResta = operacion(10, 5, ::resta)
```

# Funciones lambda

En Kotlin, puedes definir funciones anónimas conocidas como funciones lambda. Las funciones lambda son funciones sin nombre que puedes pasar como argumentos a otras funciones.

```
val suma = { a: Int, b: Int -> a + b }
```

```
val resta = { a: Int, b: Int -> a - b }
```

```
val resultadoSuma = suma(10, 5)
```

```
val resultadoResta = resta(10, 5)
```

# Los parámetros en Kotlin

A diferencia de Java, en Kotlin los parámetros de una función son inmutables por defecto, lo que significa que no se pueden modificar dentro de la función. Si necesitas modificar un parámetro dentro de una función, debes declararlo como una variable `var`.

```
fun duplicar(numero: Int): Int {  
    var resultado = numero  
    resultado *= 2  
    return resultado  
}
```

# Parámetros con valores por defecto

Los parámetros de una función en Kotlin pueden tener valores por defecto, lo que te permite llamar a la función sin proporcionar todos los argumentos.

```
fun saludar(nombre: String = "Mundo") {  
    println("Hola, $nombre!")  
}
```

```
saludar() // Hola, Mundo!  
saludar("Juan") // Hola, Juan!
```

# Funciones con valores por defecto

## Funciones con valores por defecto y su no obligatoriedad, su uso en Compose

- Las funciones con valores por defecto son muy útiles en Jetpack Compose, ya que te permiten definir componentes con valores por defecto y llamar a esos componentes sin proporcionar todos los argumentos.
- Por ejemplo, puedes definir un botón con un texto por defecto y un color por defecto, y luego llamar a ese botón sin proporcionar el texto o el color si deseas utilizar los valores por defecto.

```
@Composable
fun Boton(texto: String = "Aceptar", color: Color = Color.Blue) {
    Button(onClick = { /* Acción al hacer clic */ }) {
        Text(texto, color = color)
    }
}
```

```
Boton() // Botón con texto "Aceptar" y color azul
Boton("Cancelar", Color.Red) // Botón con texto "Cancelar" y
color rojo
```

# Parámetros de una función lambda

En una función lambda en Kotlin, puedes especificar los tipos de los parámetros de entrada o dejar que el compilador infiera los tipos automáticamente.

```
val suma: (Int, Int) -> Int = { a, b -> a + b }  
val resta = { a: Int, b: Int -> a - b }
```

*En el ejemplo anterior, se definen dos funciones lambda suma y resta que toman dos parámetros de tipo Int y devuelven un valor de tipo Int. En la función lambda suma, se especifican los tipos de los parámetros de entrada, mientras que en la función lambda resta, se deja que el compilador infiera los tipos automáticamente.*

# La palabra reservada it

En una función lambda en Kotlin, puedes utilizar la palabra reservada `it` para referirte al único parámetro de entrada si la función lambda tiene un solo parámetro.

```
val cuadrado: (Int) -> Int = { it * it }
```

*En el ejemplo anterior, se define una función lambda `cuadrado` que toma un parámetro de tipo `Int` y devuelve un valor de tipo `Int`.*

La palabra reservada `it` se utiliza para referirse al único parámetro de entrada de la función lambda.

Esto es útil cuando la función lambda tiene un solo parámetro y quieres hacer el código más conciso.

Video de refuerzo en YouTube: <https://youtube.com/shorts/h1SwVhEHAUs?si=Mqk7qXcF8qNsZMWf>

# Funciones lambda con múltiples líneas

En una función lambda en Kotlin, puedes utilizar múltiples líneas de código si es necesario.

```
val suma: (Int, Int) -> Int = { a, b ->
    val resultado = a + b
    println("La suma de $a y $b es $resultado")
    resultado
}
```

*En el ejemplo anterior, se define una función lambda suma que toma dos parámetros de tipo Int y devuelve un valor de tipo Int. La función lambda realiza la suma de los dos parámetros y muestra un mensaje por consola con el resultado.*

# Número variable de argumentos

En Kotlin, puedes definir funciones que toman un número variable de argumentos utilizando el operador **vararg**.

```
fun sumar(vararg numeros: Int): Int {  
    var suma = 0  
    for (numero in numeros) {  
        suma += numero  
    }  
    return suma  
}
```

```
val resultado = sumar(1, 2, 3, 4, 5)
```

# Funciones de extensión (Extension Functions)

En Kotlin, puedes agregar nuevas funciones a las clases existentes sin heredar de ellas. Estas funciones se conocen como funciones de extensión y te permiten extender la funcionalidad de las clases sin modificar su código fuente.

```
fun String.invertir(): String {  
    return this.reversed()  
}  
  
val texto = "Hola, mundo!"  
val textoInvertido = texto.invertir()
```

*En el ejemplo anterior, se define una función de extensión `invertir` para la clase `String` que invierte el contenido de la cadena de texto. La función de extensión se llama como si fuera un método de la clase `String`.*

SECCIÓN 5

# Null safety en Kotlin

Kotlin está diseñado para evitar los errores de referencia nula que son comunes en otros lenguajes de programación, como Java. En Kotlin, los tipos de datos pueden ser **nulos o no nulos**.

En Java, todos los tipos de datos pueden ser nulos, lo que puede llevar a errores de referencia nula si no se manejan correctamente. En Kotlin, los tipos de datos no nulos deben ser manejados de forma explícita, lo que ayuda a prevenir los errores de referencia nula.

05

# Tipos de datos nulos

Los tipos de datos pueden ser nulos o no nulos.

- **Tipos de datos no nulos:** no pueden contener valores nulos y deben ser inicializados con un valor no nulo.
- **Tipos de datos nulos:** pueden contener valores nulos y deben ser inicializados con un valor nulo o con la función null.

```
val nombre: String = "Juan" // Tipo de dato no nulo
```

```
val apellido: String? = null // Tipo de dato nulo
```

# Operadores de seguridad de nulos

En Kotlin, puedes utilizar operadores de seguridad de nulos para manejar los valores nulos de forma segura.

- **Operador de llamada segura (?.):** El operador de llamada segura ?. te permite acceder a las propiedades de un objeto nulo sin lanzar una excepción de referencia nula.
- **Operador de elvis (?:):** El operador de elvis ?: te permite proporcionar un valor predeterminado en caso de que una expresión sea nula.
- **Operador de no nulo (!!):** El operador de no nulo !! te permite forzar la ejecución de una expresión nula, lo que puede lanzar una excepción de referencia nula si la expresión es nula.

```
val nombre: String? = null
val longitud = nombre?.length
// null
val longitud = nombre?.length ?: 0
// 0
val longitud = nombre!!.length
// Lanza una excepción de
referencia nula
```

# Funciones de extensión de seguridad de nulos

En Kotlin, puedes utilizar funciones de extensión para agregar funcionalidades a los tipos de datos nulos.

```
fun String?.oLongitud(): Int {  
    return this?.length ?: 0  
}  
val nombre: String? = null  
val longitud = nombre.oLongitud() // 0
```

*En el ejemplo anterior, se define una función de extensión oLongitud para el tipo de dato String?. La función oLongitud devuelve la longitud de la cadena si no es nula, o 0 si es nula.*

## SECCIÓN 6

# POO

La programación orientada a objetos (POO) es un paradigma de programación que se basa en el concepto de "objetos", que pueden contener datos en forma de atributos y código en forma de procedimientos (métodos).

En Kotlin, puedes crear clases y objetos para modelar entidades del mundo real y encapsular datos y comportamientos relacionados. Kotlin es un lenguaje de POO y admite todas las características tradicionales de la POO, como la herencia, el polimorfismo, la encapsulación y la abstracción.

006

# Clases y Objetos

En Kotlin, puedes definir una clase utilizando la palabra clave `class` seguida del nombre de la clase y el cuerpo de la clase entre llaves `{}`.

En el ejemplo anterior, se define una clase `Persona` con dos propiedades `nombre` y `edad`. Las propiedades de la clase se inicializan con valores predeterminados.

Para crear un objeto de una clase en Kotlin, puedes utilizar la palabra clave `val` o `var` seguida del nombre del objeto, el operador de asignación `=` y la invocación del constructor de la clase.

```
class Persona {  
    var nombre: String = ""  
    var edad: Int = 0  
}
```

```
val persona = Persona()  
persona.nombre = "Juan"  
persona.edad = 30
```

# Propiedades y métodos

Puedes definir propiedades y métodos en una clase utilizando la palabra clave `var` o `val` seguida del nombre de la propiedad o el método y el tipo de dato de la propiedad o el método.

Para acceder a las propiedades y métodos de un objeto en Kotlin, puedes utilizar el operador de acceso `.` seguido del nombre de la propiedad o el método.

## DIFERENCIAS CON JAVA

A diferencia de Java, en Kotlin, las propiedades y los métodos de una clase son públicos por defecto, lo que significa que se pueden acceder desde cualquier parte del código.

```
class Persona {  
    var nombre: String = ""  
    var edad: Int = 0  
  
    fun saludar() {  
        println("Hola, soy $nombre")  
    }  
}  
  
val persona = Persona()  
persona.nombre = "Juan"  
persona.edad = 30  
persona.saludar()
```

# Encapsulación

En Kotlin, puedes encapsular propiedades y métodos en una clase utilizando los modificadores de acceso `public`, `protected`, `private` y `internal`.

- **public**: Las propiedades y métodos públicos son accesibles desde cualquier parte del código.
- **protected**: Las propiedades y métodos protegidos son accesibles desde la clase actual y las clases derivadas.
- **private**: Las propiedades y métodos privados son accesibles solo desde la clase actual.
- **internal**: Las propiedades y métodos internos son accesibles desde el módulo actual.

```
class Persona {  
    var nombre: String = ""  
    private set  
    var edad: Int = 0  
    private set  
  
    fun saludar() {  
        println("Hola, soy $nombre")  
    }  
}
```

# Encapsulación

Para acceder a las propiedades de una clase en Kotlin, puedes utilizar los métodos de acceso get y set para obtener y establecer el valor de una propiedad.

Sin embargo, al intentar modificar las propiedades nombre y edad desde fuera de la clase Persona, se produce un error de compilación debido a que las propiedades están encapsuladas con el modificador de acceso private.

```
val persona = Persona()  
persona.nombre = "Juan"  
persona.edad = 30  
persona.saludar()
```

# Constructores

Puedes definir un constructor primario utilizando la palabra clave **constructor** seguida de los parámetros del constructor.

Sin embargo, en Kotlin, puedes **omitir** la palabra clave constructor y definir los parámetros del constructor directamente en la declaración de la clase.

Para crear un objeto de una clase con un constructor primario en Kotlin, puedes utilizar la palabra clave **val** o **var** seguida del nombre del objeto, el operador de asignación **=** y la invocación del constructor de la clase con los argumentos del constructor.

```
class Persona(nombre: String, edad: Int) {  
    var nombre: String = nombre  
    var edad: Int = edad  
}
```

```
val persona = Persona("Juan", 30)
```

# Constructores Secundarios (los de Java)

Puedes definir constructores secundarios utilizando la palabra clave constructor seguida de los parámetros del constructor.

Para crear un objeto de una clase con un constructor secundario en Kotlin, puedes utilizar la palabra clave `val` o `var` seguida del nombre del objeto, el operador de asignación `=` y la invocación del constructor secundario de la clase con los argumentos del constructor.

```
class Persona {  
    var nombre: String = ""  
    var edad: Int = 0  
  
    constructor(nombre: String, edad: Int) {  
        this.nombre = nombre  
        this.edad = edad  
    }  
}  
  
val persona = Persona("Juan", 30)
```

# Herencia

En Kotlin, puedes crear una clase que herede de otra clase utilizando la palabra clave `:` seguida del nombre de la clase base.

Para crear un objeto de una clase derivada en Kotlin, puedes utilizar la palabra clave `val` o `var` seguida del nombre del objeto, el operador de asignación `=` y la invocación del constructor de la clase derivada.

```
open class Persona {  
    var nombre: String = ""  
    var edad: Int = 0  
}
```

```
class Empleado : Persona() {  
    var salario: Double = 0.0  
}
```

```
val empleado = Empleado()  
empleado.nombre = "Juan"  
empleado.edad = 30  
empleado.salario = 1000.0
```

# Polimorfismo

Puedes utilizar el polimorfismo para tratar un objeto de una clase derivada como un objeto de la clase base.

Para utilizar el polimorfismo en Kotlin, puedes crear un objeto de la clase derivada y asignarlo a una variable de la clase base.

## La palabra reservada open

La palabra reservada open se utiliza para marcar una clase o un miembro de una clase como "sobrescribible", lo que significa que puede ser heredado y sobrescrito por clases derivadas.

```
open class Persona {  
    open fun saludar() {  
        println("Hola, soy una persona")  
    }  
}  
class Empleado : Persona() {  
    override fun saludar() {  
        println("Hola, soy un empleado")  
    }  
}
```

```
val persona: Persona = Empleado()  
persona.saludar()
```

# Abstracción

Puedes utilizar la abstracción para definir una clase base con métodos abstractos que deben ser implementados por las clases derivadas.

Para utilizar la abstracción en Kotlin, puedes crear un objeto de la clase derivada y asignarlo a una variable de la clase base.

```
abstract class Persona {  
    abstract fun saludar()  
}  
  
class Empleado : Persona() {  
    override fun saludar() {  
        println("Hola, soy un empleado")  
    }  
}  
  
val persona: Persona = Empleado()  
persona.saludar()
```

# Objetos anónimos

En Kotlin, puedes crear objetos anónimos utilizando la palabra clave **object**.

Los objetos anónimos son instancias de una clase anónima que no tienen un nombre y se utilizan para definir una clase de forma concisa y reutilizable.

Los objetos anónimos son útiles cuando necesitas crear una instancia de una clase de forma rápida y concisa sin tener que definir una clase con nombre.

*En el ejemplo se crea un objeto anónimo que tiene dos propiedades nombre y edad. El objeto anónimo se asigna a la variable persona y se puede acceder a sus propiedades utilizando la notación de punto.*

```
val persona = object {  
    val nombre = "Juan"  
    val edad = 25  
}
```

```
println(persona.nombre) // Juan  
println(persona.edad) // 25
```

# Uso de objetos anónimos

Los objetos anónimos se utilizan en Kotlin para:

- Crear instancias de una clase de forma rápida y concisa.
- Definir clases de forma reutilizable sin tener que definir una clase con nombre.
- Implementar interfaces y clases abstractas de forma anónima.

Los objetos anónimos son una característica poderosa de Kotlin que te permite escribir código de forma más concisa y eficiente.

# Implementación de interfaces con objetos anónimos

Puedes implementar interfaces de forma anónima utilizando objetos anónimos. Esto te permite definir una clase que implementa una interfaz sin tener que definir una clase con nombre.

*En el ejemplo, se define una interfaz Saludable con un método saludar. Se crea un objeto anónimo que implementa la interfaz Saludable y se asigna a la variable persona. El objeto anónimo define la implementación del método saludar y se puede llamar al método utilizando la notación de punto.*

```
interface Saludable {  
    fun saludar()  
}
```

```
val persona = object : Saludable {  
    override fun saludar() {  
        println("Hola, soy una persona saludable!")  
    }  
}
```

```
persona.saludar() // Hola, soy una persona saludable!
```

# Implementación del patrón Singleton con objetos anónimos

Puedes implementar el patrón Singleton utilizando objetos anónimos. El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia.

*En el ejemplo, se define un objeto anónimo Configuración que tiene dos propiedades servidor y puerto.*

*El objeto anónimo se utiliza para almacenar la configuración de la aplicación y garantiza que solo haya una instancia de la configuración en toda la aplicación.*

```
object Configuración {  
    val servidor = "localhost"  
    val puerto = 8080  
}
```

```
println(Configuración.servidor) // localhost  
println(Configuración.puerto) // 8080
```

## SOBRE EL PATRÓN SINGLETON

Es un patrón de diseño que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia. En Kotlin, puedes implementar el patrón Singleton utilizando objetos anónimos, que garantizan que solo haya una instancia de la clase en toda la aplicación.

**Usos comunes: creación de objetos de configuración, conexiones a BBDD y puntos de acceso a servicios globales.**

SECCIÓN 6

# Clases Especiales

06

# Data classes

En Kotlin, puedes crear clases de datos utilizando la palabra clave `data`. Las clases de datos son una forma concisa de definir clases que contienen datos y no tienen lógica adicional.

```
data class Persona(val nombre: String, val edad: Int)
```

En el ejemplo anterior, se define una clase de datos `Persona` que tiene dos propiedades `nombre` y `edad`. La clase de datos se crea utilizando la palabra clave `data` seguida del nombre de la clase y las propiedades de la clase.

# Propiedades de las clases de datos

Las clases de datos en Kotlin tienen las siguientes propiedades:

- **Propiedades de solo lectura:** Las propiedades de una clase de datos son de solo lectura y no se pueden modificar una vez que se han inicializado.
- **Métodos `toString()`, `equals()` y `hashCode()`:** Las clases de datos en Kotlin generan automáticamente los métodos `toString()`, `equals()` y `hashCode()` a partir de las propiedades de la clase.
- **Métodos `componentN()`:** Las clases de datos en Kotlin generan automáticamente métodos `componentN()` que te permiten desestructurar objetos de la clase en variables individuales.
- **Método `copy()`:** Las clases de datos en Kotlin generan automáticamente un método `copy()` que te permite crear copias de objetos de la clase con propiedades modificadas.

# Objetos de clases de datos

Para crear un objeto de una clase de datos en Kotlin, puedes utilizar la palabra clave `data` seguida del nombre de la clase y los valores de las propiedades.

```
val persona = Persona("Juan", 25)
```

Puedes desestructurar objetos de clases de datos en variables individuales utilizando la notación de desestructuración.

```
val (nombre, edad) = persona
```

Puedes crear copias de objetos de clases de datos utilizando el método `copy()`.

```
val persona2 = persona.copy(edad = 30)
```

SECCIÓN 7

# Clases Enum

07

# Enum classes

Puedes crear enumeraciones utilizando la palabra clave `enum class`.

```
enum class Color {  
    ROJO, VERDE, AZUL  
}
```

Las enumeraciones son una forma de definir un conjunto fijo de constantes que representan valores específicos. Cada constante en una enumeración tiene un nombre y un valor asociado.

## Propiedades de las enumeraciones

- **Constantes:** Las enumeraciones contienen un conjunto fijo de constantes que representan valores específicos.
- **Propiedades y métodos:** Las enumeraciones pueden tener propiedades y métodos asociados con cada constante.
- **Constructores:** Las enumeraciones pueden tener constructores personalizados para inicializar las constantes con valores específicos.
- **Implementación de interfaces:** Las enumeraciones pueden implementar interfaces y heredar de clases.

# Enum Classes

## Acceso a las constantes de una enumeración

Puedes acceder a las constantes de una enumeración utilizando la notación de punto.

```
val color = Color.ROJO
```

## Propiedades y métodos de una enumeración

```
enum class Color(val rgb: Int) {  
    ROJO(0xFF0000),  
    VERDE(0x00FF00),  
    AZUL(0x0000FF);  
  
    fun nombreEnMayusculas(): String {  
        return name.toUpperCase()  
    }  
}
```

# Enum Classes en las interfaces

Las enumeraciones en Kotlin pueden implementar interfaces para agregar funcionalidades a las constantes.

```
interface Describable {  
    fun describir(): String  
}  
  
enum class DiaSemana : Describable {  
    LUNES {  
        override fun describir(): String {  
            return "Primer día de la semana"  
        }  
    },  
    MARTES {  
        override fun describir(): String {  
            return "Segundo día de la semana"  
        }  
    },  
    ...  
}
```

SECCIÓN 8

# Arrays o vectores

08

# Arrays

Puedes crear arrays utilizando la función `arrayOf()`. Los arrays en Kotlin son inmutables por defecto, lo que significa que no puedes modificar su tamaño una vez que se han creado.

```
val numeros = arrayOf(1, 2, 3, 4, 5)
```

Puedes acceder a elementos individuales de un array utilizando el operador de indexación `[]` seguido del índice del elemento que deseas acceder.

```
val primerNumero = numeros[0]
```

```
val segundoNumero = numeros[1]
```

Para modificar un elemento de un array, puedes utilizar el operador de indexación `[]` seguido del índice del elemento que deseas modificar.

```
numeros[0] = 10
```

# Arrays

## Iteración sobre un array

Puedes iterar sobre los elementos de un array utilizando un bucle for o la función `forEach()`.

```
for (numero in numeros) {  
    println(numero)  
}
```

```
numeros.forEach { numero ->  
    println(numero)  
}
```

# Arrays

## Arrays de tipos primitivos

Puedes utilizar arrays de tipos primitivos como `IntArray`, `DoubleArray`, `BooleanArray`, etc. para mejorar el rendimiento de tu código.

```
val enteros = intArrayOf(1, 2, 3, 4, 5)
val dobles = doubleArrayOf(1.0, 2.0, 3.0, 4.0, 5.0)
val booleanos = booleanArrayOf(true, false, true, false)
```

## Operaciones con arrays

Los arrays en Kotlin proporcionan una serie de operaciones útiles:

```
val tamaño = numeros.size
val contieneTres = numeros.contains(3)
val indiceDeCuatro = numeros.indexOf(4)
```

SECCIÓN 9

**Lists o listas**

09

# Listas

Crear listas utilizando la función `listOf()`. Las listas en Kotlin son inmutables por defecto, lo que significa que no puedes modificar su tamaño una vez que se han creado.

```
val numeros = listOf(1, 2, 3, 4, 5)
```

Acceso a elementos:

```
val primerNumero = numeros[0]  
val segundoNumero = numeros[1]
```

# Listas

## Iteración

Puedes iterar sobre los elementos de una lista utilizando un bucle for o la función `forEach()`.

```
for (numero in numeros) {  
    println(numero)  
}
```

```
numeros.forEach { numero ->  
    println(numero)  
}
```

# Listas

## Listas de tipos primitivos

Puedes utilizar listas de tipos primitivos como `IntList`, `DoubleList`, `BooleanList`, etc. para mejorar el rendimiento del código.

```
val enteros = intListOf(1, 2, 3, 4, 5)
val dobles = doubleListOf(1.0, 2.0, 3.0, 4.0, 5.0)
val booleanos = booleanListOf(true, false, true, false)
```

## Modificación de elementos

Las listas en Kotlin son inmutables por defecto, por lo que no puedes modificar los elementos de una lista una vez que se ha creado. Si necesitas una lista mutable, puedes utilizar la función `mutableListOf()`.

```
val numeros = mutableListOf(1, 2, 3, 4, 5)
numeros[0] = 10
```

# Listas - Operaciones comunes I

Kotlin proporciona una serie de funciones de extensión para realizar operaciones comunes con listas:

```
val cuadrados = numeros.map { numero -> numero * numero }  
val pares = numeros.filter { numero -> numero % 2 == 0 }  
val suma = numeros.reduce { acc, numero -> acc + numero }  
val ordenados = numeros.sortedBy { numero -> numero }
```

- **map()**: Aplica una función a cada elemento de la lista y devuelve una nueva lista con los resultados.
- **filter()**: Filtra los elementos de la lista que cumplen una condición dada y devuelve una nueva lista con los elementos filtrados.
- **reduce()**: Combina los elementos de la lista en un solo valor utilizando una función dada.
- **sortedBy()**: Ordena los elementos de la lista en función de una clave dada y devuelve una nueva lista ordenada.
- **groupBy()**: Agrupa los elementos de la lista en función de una clave dada y devuelve un mapa de claves y listas de elementos.

# Listas - Operaciones comunes II

```
val suma = numeros.sum()
val maximo = numeros.max()
val minimo = numeros.min()
val promedio = numeros.average()
val unicos = numeros.distinct()
```

- **sum()**: Calcula la suma de los elementos de la lista.
- **max()**: Devuelve el valor máximo de la lista.
- **min()**: Devuelve el valor mínimo de la lista.
- **average()**: Calcula el promedio de los elementos de la lista.
- **distinct()**: Devuelve una nueva lista con los elementos únicos de la lista original.

# Listas

## Funciones de extensión

Puedes crear tus propias funciones de extensión para realizar operaciones específicas con listas.

```
fun List<Int>.duplicar(): List<Int> {  
    return this.map { it * 2 }  
}  
  
val duplicados = numeros.duplicar()
```

SECCIÓN 10

# Maps o Diccionarios

10

# Maps

un Map es una colección de pares clave-valor donde cada clave está asociada a un valor. Los Map en Kotlin son inmutables por defecto, lo que significa que no puedes modificar su tamaño una vez que se han creado.

```
val numeros = mapOf("uno" to 1, "dos" to 2, "tres" to 3)
```

Puedes acceder a elementos individuales de un Map utilizando la **clave** del elemento:

```
val numeroUno = numeros["uno"]  
val numeroDos = numeros["dos"]
```

# Maps

Iteración con for y forEach():

```
for ((clave, valor) in numeros) {  
    println("Clave: $clave, Valor: $valor")  
}
```

```
numeros.forEach { (clave, valor) ->  
    println("Clave: $clave, Valor: $valor")  
}
```

# Maps

## Maps de tipos primitivos (para mejorar rendimiento)

```
val enteros = intMapOf(1 to "uno", 2 to "dos", 3 to "tres")  
val dobles = doubleMapOf(1.0 to "uno", 2.0 to "dos", 3.0 to "tres")  
val booleanos = booleanMapOf(true to "verdadero", false to "falso")
```

## Modificación de elementos:

Los Map en Kotlin son inmutables por defecto, por lo que no puedes modificar los elementos de un Map una vez que se ha creado. Si necesitas un Map mutable, puedes utilizar la función mutableMapOf().

```
val numeros = mutableMapOf("uno" to 1, "dos" to 2, "tres" to 3)  
numeros["uno"] = 10
```

# Maps Operaciones comunes

- `get()`: Obtiene el valor asociado a una clave.
- `containsKey()`: Comprueba si una clave está presente en el Map.
- `containsValue()`: Comprueba si un valor está presente en el Map.
- `keys`: Obtiene las claves del Map.
- `values`: Obtiene los valores del Map.
- `filterKeys()`: Filtra los pares clave-valor del Map por las claves.
- `filterValues()`: Filtra los pares clave-valor del Map por los valores.

```
val valorUno = numeros.get("uno")
val contieneDos =
numeros.containsKey("dos")
val contieneCinco =
numeros.containsValue(5)
val claves = numeros.keys
val valores = numeros.values
val numerosPares = numeros.filterKeys {
clave -> clave.length % 2 == 0 }
val numerosImpares = numeros.filterValues {
valor -> valor % 2 != 0 }
```

SECCIÓN 11

# Sets o Conjuntos



# Sets

Un Set es una colección de elementos únicos, lo que significa que no puede contener elementos duplicados. Los Set en Kotlin son inmutables por defecto, lo que significa que no puedes modificar su tamaño una vez que se han creado.

```
val numeros = setOf(1, 2, 3, 4, 5)
```

## Acceso a elementos:

Puedes acceder a elementos individuales de un Set utilizando la función `contains()` para comprobar si un elemento está presente en el Set.

```
val contieneTres = numeros.contains(3)  
val contieneSeis = numeros.contains(6)
```

# Sets

CONCEPTO

Iteración con for y forEach():

```
for (numero in numeros) {  
    println(numero)  
}
```

```
numeros.forEach { numero -> println(numero) }
```

# Sets

## Sets de tipos primitivos:

En Kotlin, puedes utilizar Set de tipos primitivos como IntSet, DoubleSet, BooleanSet, etc. para mejorar el rendimiento de tu código.

```
val enteros = intSetOf(1, 2, 3, 4, 5)
val dobles = doubleSetOf(1.0, 2.0, 3.0, 4.0, 5.0)
val booleanos = booleanSetOf(true, false, true, false)
```

## Modificación de elementos:

Los Set en Kotlin son inmutables por defecto, por lo que no puedes modificar los elementos de un Set una vez que se ha creado. Si necesitas un Set mutable, puedes utilizar la función mutableSetOf().

```
val numeros = mutableSetOf(1, 2, 3, 4, 5)
numeros.add(6)
```

# Sets – Operaciones comunes

- **union()**: Devuelve un Set que contiene todos los elementos de los dos Set.
- **intersect()**: Devuelve un Set que contiene los elementos comunes de los dos Set.
- **subtract()**: Devuelve un Set que contiene los elementos del primer Set que no están en el segundo Set.
- **containsAll()**: Comprueba si un Set contiene todos los elementos de otro Set.
- **isEmpty()**: Comprueba si un Set está vacío.

```
val numerosPares = setOf(2, 4, 6, 8, 10)
val numerosImpares = setOf(1, 3, 5, 7, 9)

val union = numeros.union(numerosPares)
val interseccion = numeros.intersect(numerosImpares)
val diferencia = numeros.subtract(numerosPares)
val contieneTodos = numeros.containsAll(numerosPares)
val esVacio = numeros.isEmpty()
```

SECCIÓN 12

# Usos Avanzados

12

# Genéricos en clases y funciones

En Kotlin, los objetos genéricos te permiten escribir código que es reutilizable y flexible, ya que puedes definir clases y funciones que trabajen con cualquier tipo de datos. Puedes utilizar genéricos en clases, funciones e interfaces para crear código genérico y flexible.

```
interface Contenedor<T> {  
    fun obtenerContenido(): T  
}
```

```
fun <T : Number> sumar(a: T, b: T): T {  
    return a + b  
}
```

```
class Caja<T>(val contenido: T) {  
    fun obtenerContenido(): T {  
        return contenido  
    }  
}
```

```
fun <T> imprimirElemento(elemento: T) {  
    println(elemento)  
}
```

# Funciones Scope

Los scope functions son funciones que te permiten ejecutar un bloque de código en el contexto de un objeto. Estas funciones te permiten acceder a las propiedades y métodos del objeto de forma más concisa y legible.

**Let:** permite ejecutar un bloque de código en el contexto de un objeto y devolver el resultado de la última expresión del bloque.

|                         | { .. this .. } | { .. it .. } |
|-------------------------|----------------|--------------|
| return result of lambda | with / run     | let          |
| return receiver         | apply          | also         |

  

|                                                 |                                                  |
|-------------------------------------------------|--------------------------------------------------|
| receiver.apply {<br><b>this</b> .actions()<br>} | receiver.also {<br>moreActions( <b>it</b> )<br>} |
|-------------------------------------------------|--------------------------------------------------|

```
val resultado = persona?.let {  
    println("Nombre: ${it.nombre}")  
    println("Edad: ${it.edad}")  
    it.edad + 1  
}
```

# Funciones Scope

**Run:** A diferencia de let, la función run te permite acceder a las propiedades y métodos del objeto sin necesidad de utilizar it.

**With:** La función with es similar a run, pero se utiliza con un objeto como argumento en lugar de un receptor.

```
val resultado = persona?.run {  
    println("Nombre: $nombre")  
    println("Edad: $edad")  
    edad + 1  
}
```

```
val resultado = with(persona) {  
    println("Nombre: $nombre")  
    println("Edad: $edad")  
    edad + 1  
}
```

# Funciones Scope

**Apply:** A diferencia de las funciones anteriores, la función `apply` se utiliza para realizar operaciones en un objeto y devolver el objeto modificado.

```
val resultado = persona?.apply {  
    println("Nombre: $nombre")  
    println("Edad: $edad")  
    edad += 1  
}
```

**Also:** La función `also` es similar a `apply`, pero se utiliza con un objeto como argumento en lugar de un receptor.

```
val resultado = persona?.also {  
    println("Nombre: ${it.nombre}")  
    println("Edad: ${it.edad}")  
}
```

# Clases selladas

Las clases selladas son un tipo especial de clase en Kotlin que se utilizan para representar un conjunto finito de subclases.

Las clases selladas son útiles cuando tienes un conjunto limitado de subclases y quieres asegurarte de que todas las subclases se manejan de forma segura en un bloque **when**.

# Clases selladas

## Declaración

Puedes declarar una clase sellada utilizando la palabra clave sealed antes de la palabra clave class.

```
sealed class Resultado {  
    data class Exito(val mensaje: String) : Resultado()  
    data class Error(val mensaje: String) : Resultado()  
}
```

## Uso:

Las clases selladas se utilizan principalmente en expresiones when para manejar de forma segura todas las subclases.

```
fun procesarResultado(resultado: Resultado) {  
    when (resultado) {  
        is Resultado.Exito -> println("Exit: ${resultado.mensaje}")  
        is Resultado.Error -> println("Error: ${resultado.mensaje}")  
    }  
}
```

# Bibliografía

CONCEPTO

Documentación oficial: <https://kotlinlang.org/api/core/kotlin-stdlib/>

# Recursos bibliográficos

Baeldung. (s. f.). Kotlin overview. <https://www.baeldung.com/kotlin/kotlin-overview>McGregor, D., & Pryce, N. (2021).

Java to Kotlin: A refactoring guidebook. Pragmatic Bookshelf.MindOrks Open Source. (s. f.). From Java to Kotlin (README-ES).

GitHub. <https://github.com/MindorksOpenSource/from-java-to-kotlin/blob/master/README-ES.md>Subramaniam, V. (2019).

Programming Kotlin. Pragmatic Bookshelf.The Kotlin Foundation. (s. f.). Kotlin. <https://kotlinlang.org/>

ResuaCode. (s. f.). Kotlin apuntes. GitHub. <https://github.com/resuacode/kotlin-apuntes>



# ¿Preguntas?

*Espacio abierto para dudas, comentarios o reflexiones sobre lo visto hoy*